RaceDay Event Management System

PROG6212 Programming 2B — PoE Part 2

RaceDay is a web-based event management API for South African road running, walking and cycling events. It supports account registration, session-based authentication, event management, event categories, participant enrolments and race results.

This repository extends the planning and database work from Part 1 with an ASP.NET Core Web API and an automated build-and-test workflow.

Technology Stack

- C# and .NET 8
- ASP.NET Core Web API with controllers
- Entity Framework Core 8
- SQL Server and LocalDB for local development
- ASP.NET Core session state
- ASP.NET Core password hashing
- Swagger / OpenAPI
- xUnit for automated testing
- GitHub Actions for continuous integration

Project Structure

```
RaceDay/
├── .github/
│   └── workflows/
│       ├── part1-ci.yml
│       └── part2-ci.yml
├── RaceDay.Api/
│   ├── Controllers/
│   ├── Data/
│   ├── Migrations/
```

```
│   ├── Models/
│   ├── Security/
│   ├── Program.cs
│   └── appsettings.json
├── RaceDay.Api.Tests/
│   ├── AuthControllerTests.cs
│   ├── RaceDayWebApplicationFactory.cs
│   └── UnitTest1.cs
├── RaceDay.Contracts/
│   └── Dtos.cs
├── docs/
├── RaceDay.slnx
└── README.md
```

Prerequisites

Install the following before running the application:

- .NET 8 SDK
- Visual Studio 2022 with the ASP.NET and web development workload, or a compatible .NET development environment
- SQL Server LocalDB on Windows for the configured local database
- Git

Getting Started

Clone the repository and check out the Part 2 branch:

```
git clone https://github.com/Derrick121K/RaceDay.git
cd RaceDay
git checkout part2-api-development
```

Open `RaceDay.slnx` in Visual Studio.

Database Configuration

The API uses the `DefaultConnection` connection string in `RaceDay.Api/appsettings.json`.

The local development configuration uses SQL Server LocalDB and the `RaceDayPart2` database. Keep this separate from the Part 1 database.

Before running the API for the first time, apply the Entity Framework Core migration.

In Visual Studio's Package Manager Console, select `RaceDay.Api` as the Default project and run:

```
Update-Database
```

Alternatively, after installing the compatible EF Core command-line tool, run:

```
dotnet ef database update --project RaceDay.Api --startup-project RaceDay.Api
```

Run the API

Start `RaceDay.Api` using Visual Studio or run:

```
dotnet run --project RaceDay.Api/RaceDay.Api.csproj
```

Open the Swagger UI at the HTTPS URL shown in the application output, followed by `/swagger`.

For example, during local development:

`https://localhost:7155/swagger/index.html`

Your port may differ depending on the launch profile.

Authentication and Session Management

RaceDay uses server-side session state.

- Successful login stores the user's `UserId` and `Role` in the session.
- Passwords are hashed using ASP.NET Core's password hasher.
- Passwords and password hashes are not returned in authentication responses.
- Logout clears the session.
- Protected actions use the session authorization attribute to check authentication and, where required, the user's role.

The local development application uses an in-memory distributed cache for session storage. This is suitable for development and is not a persistent, shared session store for a multi-instance production deployment.

User Roles

Organiser

Organisers can create events, manage events they own, manage their event categories, view enrolments for their events, update enrolment statuses and record or maintain results for their events.

Participant

Participants can register, log in, browse events, enrol in events, view their own enrolments and view their own results.

The API checks the session identity and role for protected operations. Organiser event-management actions also check ownership of the relevant event.

API Endpoint Reference

All request and response schemas can also be inspected in Swagger while the API is running.

Authentication

Method	Route	Purpose
POST	`/api/auth/register`	Register an account
POST	`/api/auth/login`	Authenticate an account and start a session
POST	`/api/auth/logout`	Clear the current session
GET	`/api/auth/me`	Return the current session user's details

Events

Method	Route	Purpose	Access
GET	`/api/events`	List events	Public
GET	`/api/events/{id}`	View an event	Public
POST	`/api/events`	Create an event	Organiser

Method	Route	Purpose	Access
PUT	`/api/events/{id}`	Update an owned event	Organiser / owner
DELETE	`/api/events/{id}`	Delete an owned event	Organiser / owner

Categories

Method	Route	Purpose	Access
GET	`/api/events/{eventId}/categories`	List categories for an event	Public
GET	`/api/categories/{id}`	View a category	Public
POST	`/api/events/{eventId}/categories`	Create a category	Organiser / owner
PUT	`/api/categories/{id}`	Update a category	Organiser / owner
DELETE	`/api/categories/{id}`	Delete a category	Organiser / owner

Enrolments

Method	Route	Purpose	Access
POST	`/api/enrolments`	Enrol in an event and select a category	Participant
GET	`/api/enrolments/me`	List the current participant's enrolments	Participant
GET	`/api/events/{eventId}/enrolments`	List enrolments for an owned event	Organiser / owner
PUT	`/api/enrolments/{id}/status`	Update an enrolment status for an owned event	Organiser / owner

The API checks that a selected category belongs to the chosen event and prevents a participant from enrolling in the same event more than once.

Results

Method	Route	Purpose	Access
POST	`/api/enrolments/{enrolmentId}/result`	Record a finishing time and position	Organiser / owner
GET	`/api/results/me`	View the current participant's results	Participant
GET	`/api/enrolments/{enrolmentId}/result`	View a result when authorised	Participant / enrolment owner or Organiser / event owner
PUT	`/api/enrolments/{enrolmentId}/result`	Update a result for an owned event	Organiser / owner
DELETE	`/api/enrolments/{enrolmentId}/result`	Delete a result for an owned event	Organiser / owner

Running Automated Tests

The test project uses xUnit, ASP.NET Core's test host and Entity Framework Core's in-memory provider.

The test factory replaces the normal SQL Server registration with a separate in-memory database for API tests.

Run the tests from the repository root:

```
dotnet restore RaceDay.Api.Tests/RaceDay.Api.Tests.csproj
dotnet build RaceDay.Api.Tests/RaceDay.Api.Tests.csproj --configuration Release
dotnet test RaceDay.Api.Tests/RaceDay.Api.Tests.csproj --configuration Release
```

The automated tests currently cover:

- Registration request validation
- Invalid registration returning `400 Bad Request`
- Successful registration returning `201 Created`
- Duplicate email registration returning `409 Conflict`
- Successful login returning user details
- Rejection of an incorrect password with `401 Unauthorized`

- Logout returning `204 No Content`

These tests do not constitute complete automated coverage of every event, category, enrolment and result endpoint. Those operations should be verified with additional endpoint-specific tests.

Continuous Integration

The Part 2 workflow is located at:

`.github/workflows/part2-ci.yml`

It runs on pushes and pull requests targeting `main` or `part2-api-development`. The workflow:

1. Checks out the repository.
2. Sets up the .NET 8 SDK.
3. Restores dependencies.
4. Builds the API and its test project.
5. Runs the automated tests.

The latest recorded Part 2 workflow run succeeded:

View the GitHub Actions run

Part 1 Documentation

The original database design, SQL script, ERD and API endpoint plan remain in the `docs/` directory. They provide the design reference for the Part 2 implementation.

Video Demonstration

Part 2 demonstration: Add the Unlisted YouTube link for the Part 2 demonstration before final submission.

The existing Part 1 presentation is available here:

RaceDay PROG6212 Part 1 Presentation

Repository and Submission

Repository:

https://github.com/Derrick121K/RaceDay

Part 2 development branch:

https://github.com/Derrick121K/RaceDay/tree/part2-api-development

Submit the repository or branch link and the required Part 2 demonstration link using the assessment submission instructions.